



EAGLE-3: 通过训练时测试扩展大语言模型的推理加速

Yuhui Li^{1,3}, Fangyun Wei², Chao Zhang¹, Hongyang Zhang^{3,4}

¹ 北京大学 ² 微软研究院

³ 滑铁卢大学 ⁴ Vector Institute

yuhui.li@stu.pku.edu.cn, fawe@microsoft.com

c.zhang@pku.edu.cn, hongyang.zhang@uwaterloo.ca

Abstract

现代大语言模型 (LLM) 的顺序生成特性使其推理代价高昂且速度缓慢，而推测性采样 (speculative sampling) 已被证明是解决该问题的有效方案。EAGLE 等方法在特征层面执行自回归，通过复用目标模型的顶层特征来取得比原始推测性采样更优的效果。LLM 社区中日益兴起的趋势是通过扩大训练数据来提升模型智能，同时不增加推理成本。然而，我们观察到扩大数据规模对 EAGLE 的提升十分有限。我们发现，这一局限源于 EAGLE 的特征预测约束。本文提出 EAGLE-3，它放弃了特征预测，转而直接预测词元 (token)，并通过名为“训练时测试”的技术，以多层特征融合取代对顶层特征的依赖。这些改进显著提升了性能，使草稿模型能够充分受益于训练数据的扩大。我们的实验涵盖对话模型和推理模型，在五项任务上进行评估。结果表明，EAGLE-3 可实现高达 6.5 倍的加速比，比 EAGLE-2 提升约 1.4 倍。在 SGLang 框架中，EAGLE-3 在批大小为 64 时实现了 1.38 倍的吞吐量提升。代码已开源：<https://github.com/SafeAILab/EAGLE>。

1 引言

现代大语言模型正被应用于越来越多的领域，其能力的提升主要依赖于模型参数的扩大——部分 LLM 现已超过数千亿参数。在自回归生成过程中，每个词元都需要访问所有模型参数，使得 LLM 推理速度缓慢且成本高昂。

近年来，测试时扩展受到了广泛关注。ChatGPT o1 和 DeepSeek-R1 (Guo et al., 2025) 等模型在回答前会进行深思熟虑的推理，以推理时间换取更强的 LLM 能力。然而，这些模型往往需要漫长的推理过程，导致成本极高，而响应时间的增加也严重影响用户体验。推理模型显著提升了推理成本在整个 LLM 流水线中的占比，推动研究者积极探索更廉价、更快速的推理优化方法。

推测性采样方法可通过部分并行化生成过程来降低 LLM 延迟。这些方法快速生成草稿词

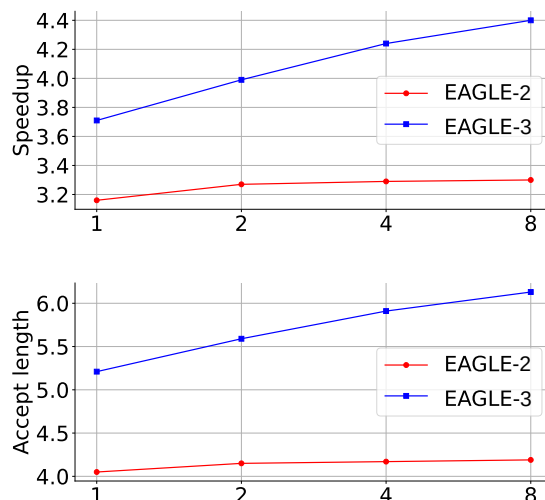


Figure 1: 以 LLaMA-Instruct 3.1 8B 为目标模型在 MT-bench 上评估的扩展规律，横轴为相对于 ShareGPT 的数据规模。EAGLE-3 的新架构设计实现了单调递增的扩展曲线，这在此前的工作中从未观察到。

元，然后并行验证它们，从而在单次前向传播中生成多个词元，显著降低推理延迟。在原始推测性采样中，草稿模型是与目标模型同系列的较小型 LLM，独立运行。与原始推测性采样不同，EAGLE (Li et al., 2024c) 复用目标模型的顶层特征 (LM head 之前的特征)，训练草稿模型自回归地预测下一个特征，再利用目标模型的 LM head 获得草稿词元。借助目标模型的丰富信息，EAGLE 相比原始推测性采样取得了更显著的加速效果。HASS (Zhang et al., 2024) 和 Falcon (Gao et al., 2024) 等后续方法也采用了利用当前特征序列预测下一特征的思路。

近期 LLM 越来越依赖更大的训练数据集来提升性能。例如，7B (8B) 规模的 LLaMA 系列模型分别使用了 1T、2T 和 15T 词元的训练数据 (LLaMA 1 (Touvron et al., 2023a)、LLaMA 2 (Touvron et al., 2023b)、LLaMA 3 (Dubey et al., 2024))，在保持模型架构和推理成本基本不变的情况下，各项指标均取得了显著提升。同样地，我们希望通过增加训练数据来提升

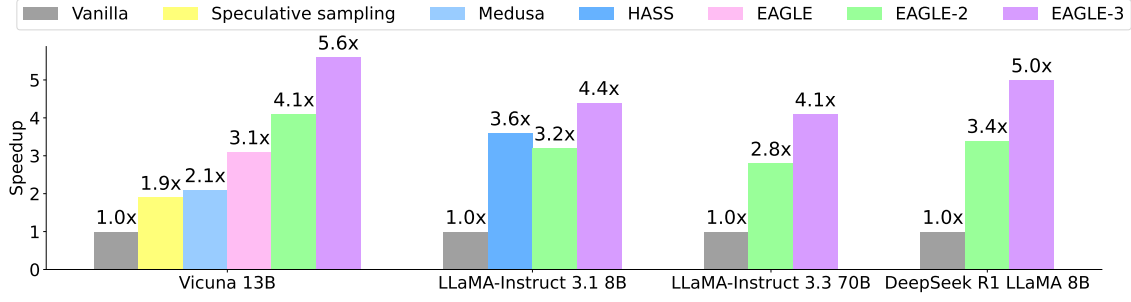


Figure 2: 不同方法在温度 =0 时的加速比。对于标准推测性采样，Vicuna-13B 使用 Vicuna-68M 作为草稿模型。表 1 还提供了与其他方法的对比，但本图仅展示部分方法。对话模型的评测数据集为 MT-bench，推理模型的评测数据集为 GSM8K。DeepSeek R1 LLaMA 8B 指 DeepSeek-R1-Distill-LLaMA 8B。

升 EAGLE 的接受率和加速比。遗憾的是，我们观察到额外训练数据对 EAGLE 的收益十分有限。

我们对这一现象的原因进行了分析。如图 3 上部所示，EAGLE 在特征层面执行自回归预测，预测下一个特征后再输入目标模型的 LM head 获取词元分布。EAGLE 的损失函数包含两个部分：特征预测损失 l_{fea} 和词元预测损失 l_{token} 。得益于特征预测损失，仅在第一步训练的草稿模型可以适应第二步，并获得多步预测能力。然而，以词元预测为最终目标时，特征预测可视为一种额外约束，限制了草稿模型的表达能力，使其难以从更多数据中获益。去除特征约束并扩大训练数据后（图 3 中部），如图 4 所示，第一个草稿词元的接受率 $0-\alpha$ 显著提升。然而，草稿模型在第一步输出的 $\hat{a}_{t+1}$ 与真实值 f_{t+1} 相差较大，导致第二步输入序列 $f_1, f_2, \dots, f_t, \hat{a}_{t+1}$ 偏离训练分布，使第二个草稿词元的接受率 $1-\alpha$ 极低，如图 4 所示。我们可以通过在训练过程中引入第一步（图 3 下部）来解决这一问题。采用此方法后，增加训练数据的收益更加显著。我们将这一技术命名为**训练时测试**（training-time test）。

EAGLE 以及 Medusa (Cai et al., 2024) 等推测性采样方法复用目标模型的顶层特征，即紧邻 LM head 之前的特征。对于满秩权重矩阵的 LM head，对应下一词元 logits 的顶层特征是唯一的，确保这些特征所含信息与下一词元的 logits 直接对应。然而，仅凭本质上局限于下一词元的顶层特征来预测下下个词元，是一项巨大挑战。幸运的是，上述训练时测试技术使得使用中间层特征成为可能，无需仅依赖顶层——因为训练时已移除特征预测损失 l_{fea} 。

综上所述，本文提出 EAGLE-3，即 EAGLE 的增强版本，实现了显著的加速提升。EAGLE-3 支持并行化，并与 EAGLE-2 (Li et al., 2024b) 的草稿树技术完全兼容。本文的主要贡献如下：

- **面向草稿模型的新型训练时测试架构**：我们去除了特征预测约束，在训练过程中模拟多步生成来直接预测词元。这种直接词元预测方式为草稿模型的输入提供了完全的灵活性。我们不再仅复用顶层特征，而是整合利用目标模型的低层、中层和高层特征，从不同层次捕捉丰富的语义信息。
- **发现大语言模型推理加速的扩展规律**：在新架构下，我们观察到增加草稿模型训练数据量可以同比提升 EAGLE-3 的加速比。这一扩展行为在原始 EAGLE 架构中从未观察到，如图 1 所示。
- **更优的推理加速性能**：使用约 8 倍于 EAGLE 训练数据的 EAGLE-3，在批大小为 1 时实现了比 EAGLE-2 高 1.4 倍的延迟加速。推测性采样常被认为会降低大批大小下的吞吐量。然而，在生产级框架 SGLang (Zheng et al., 2024) 中，EAGLE-3 在批大小为 64 时吞吐量提升了 40%。我们预期更大的数据规模将带来进一步的加速比提升。

2 预备知识

2.1 推测性采样

推测性采样 (Leviathan et al., 2023; Chen et al., 2023; Sun et al., 2024c,b) 是一种无损的 LLM 加速技术，交替进行草稿生成和验证，其中草稿生成以低成本进行，验证过程并行化执行。我们用 t_i 表示第 i 个词元，用 $T_{a:b}$ 表示词元序列 $t_a, t_{a+1}, \dots, t_b$ 。以 $T_{1:j}$ 作为前缀时，推测性采样的两个阶段如下。

在**草稿阶段**，推测性采样利用草稿模型（目标模型同系列的较小版本）自回归地生成 k 个词元以构成草稿 $\hat{T}_{j+1:j+k}$ ，同时记录每个词元的概率 $\hat{p}$ 。

在**验证阶段**，推测性采样调用目标模型评估草稿 $\hat{T}_{j+1:j+k}$ 并记录其概率 p 。推测

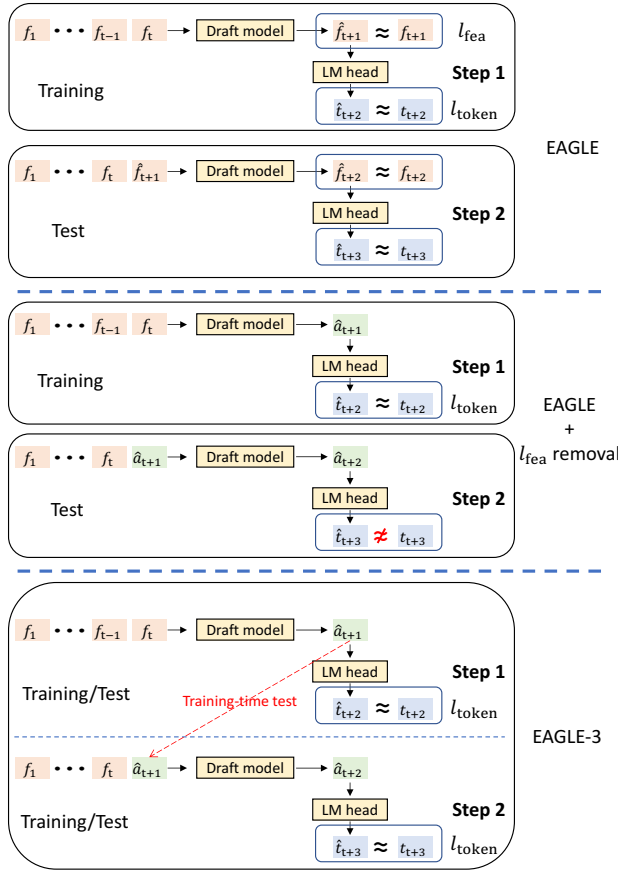


Figure 3: 训练时测试（下部）及其与其他草稿方法（上部和中部）的对比示意图。 f 表示特征， t 表示词元， a 表示无约束向量。帽符号（hat）表示模型的预测值。图中所有方法均使用前一时间步的词元序列，但为简洁起见未在图中展示。EAGLE-3 的实际输入并非 f ，详细说明见后续章节。

性采样从前到后依次判断草稿词元是否被接受。对于词元 $\hat{t}_{j+i}$ ，其被接受的概率为 $\min(1, p_{j+i}(\hat{t}_{j+i})/\hat{p}_{j+i}(\hat{t}_{j+i}))$ 。若该词元被接受，则继续处理下一个词元；否则，从分布 $\text{norm}(\max(0, p_{j+i} - \hat{p}_{j+i}))$ 中采样一个词元替换 $\hat{t}_{j+i}$ ，并丢弃草稿中剩余的词元。(Leviathan et al., 2023) 的附录 A.1 证明了推测性采样与原始自回归解码的分布一致性。

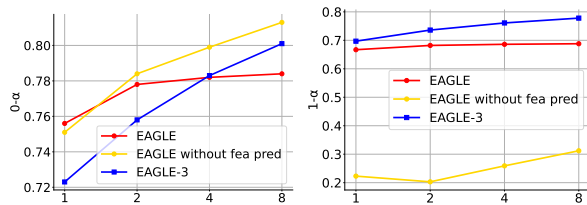


Figure 4: 不同方法的接受率对比，横轴为相对于 ShareGPT 的数据规模。

2.2 EAGLE 与 EAGLE-2

容量有限的草稿模型难以精确逼近大规模目标模型。EAGLE 将目标模型的顶层特征作为额外信息，在特征层面进行自回归，从而简化草稿生成过程。EAGLE 在特征层面执行自回归，再利用目标模型的 LM head 获取草稿词元。由于词元层面的采样结果不可见，特征级自回归引入了不确定性。EAGLE 通过将前一时间步的词元序列（即采样结果）输入草稿模型来解决这一问题。

与原始推测性采样的链式草稿不同，EAGLE 在同一位置生成多个草稿词元，形成树状草稿。在验证阶段，EAGLE 使用树形注意力机制并行验证草稿树。值得一提的是，EAGLE 启发了 DeepSeek-v3 (Liu et al., 2024a) 预训练中使用的多词元预测技术，后者又反过来启发了 EAGLE-3 的新架构设计。

EAGLE (Li et al., 2024c) 和 Medusa (Cai et al., 2024) 等方法使用树形草稿，其草稿树结构是预先定义的、静态的、与上下文无关的。草稿难度与上下文密切相关，静态草稿树可能导致资源浪费。EAGLE-2 (Li et al., 2024b) 利用草稿模型的置信度近似接受率，并据此动态生成草稿树，在草稿阶段结束时对草稿树进行剪枝。EAGLE-3 同样采用了 EAGLE-2 提出的上下文感知动态草稿树。

3 EAGLE-3

本节详细介绍 EAGLE-3 的实现方案。

3.1 推理流水线

与其他推测性采样方法一致，EAGLE-3 交替执行草稿生成和验证阶段。EAGLE-3 与 EAGLE 的区别在于草稿阶段，我们通过一个示例加以说明，如图 5 所示。考虑前缀“How can”（如何能）。

在预填充阶段或上一轮验证阶段中，目标模型执行前向传播，生成下一个词元“I”（我）。我们记录目标模型前向传播中的低层、中层和高层特征序列，分别记为 l 、 m 和 h 。我们将 k 维向量 l 、 m 、 h 拼接成 $3k$ 维向量，再通过全连接（FC）层降维至 k 维，得到融合了不同层信息的特征 g 。其中 k 为目标模型的隐藏层维度。

我们的目标是以“How can I”为前缀生成草稿词元序列。仅输入 g_{how} 和 g_{can} 时，草稿模型无法访问随机采样过程。因此，类似于 EAGLE (Li et al., 2024c)，我们引入采样词元“I”的嵌入 e_I 。拼接后的向量通过 FC 层降维至 k 维，再输入单层解码器，产生输出 a 。最后，我们将 a_I 输入 LM head 并采样得到草

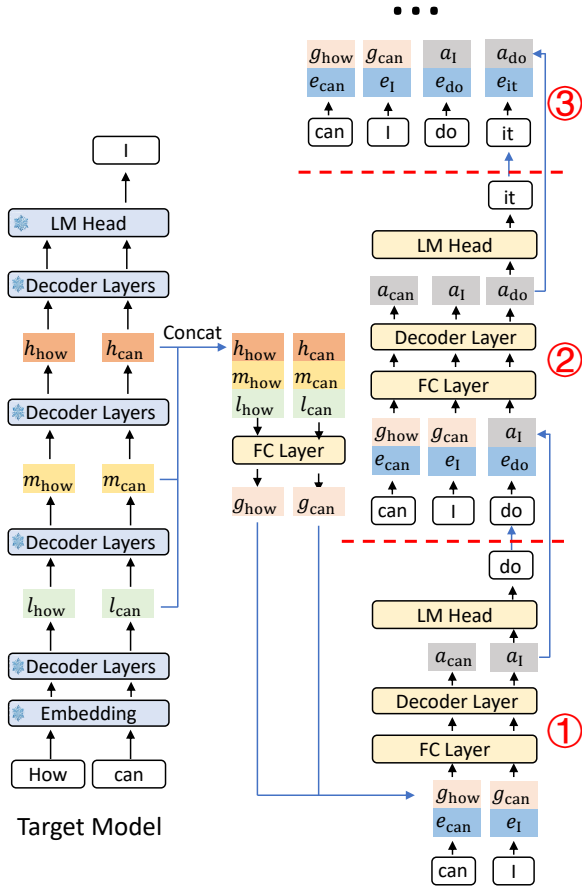


Figure 5: EAGLE-3 推理流水线示意图，展示草稿模型的三个步骤。 l 、 m 、 h 分别表示目标模型的底层、中层和高层特征， e 表示词嵌入。

稿词元”do”（做）。

在第一步中，以”How can”为前缀，我们复用目标模型的 g_{how} 和 g_{can} 。在第二步中，前缀变为”How can I”。理想情况下，我们应复用目标模型的 g_{how} 、 g_{can} 和 g_I 。然而，由于词元”I”尚未经目标模型验证，无法获得 g_I 。我们转而使用上一步草稿模型的输出 a_I 替代 g_I ，并将 a_I 与采样结果”do”的嵌入 e_{do} 拼接，作为第一步草稿模型的输入。在第三步中，同样无法获得 g_{do} ，因此使用 a_{do} 作为替代，将 a_{do} 与 e_{it} 拼接作为草稿模型的输入。后续步骤以此类推。

3.2 草稿模型训练

EAGLE 中草稿模型的输入是，或至少近似于，目标模型的顶层特征 $f_1, f_2, \dots, f_t$ 。相比之下，EAGLE-3 中草稿模型的输入既可能是目标模型的特征 $g_1, g_2, \dots, g_t$ ，也可能包含草稿模型自身的输出 $a_{t+1}, a_{t+2}, \dots, a_{t+j}$ 。因此，我们需要训练草稿模型以适应不同的输入。在训练过程中，我们执行测试步骤，生成 a 并将其反馈给草稿模型继续训练。

EAGLE-3 草稿模型的核心是一个 Trans-

former 解码器层。除自注意力操作外，其他组件均不与上下文交互，因此在训练或测试时无需进行其他修改。唯一需要轻微修改的组件是自注意力，下面详细说明。

尽管实际输入为特征，但为便于描述，我们以词元作为输入进行说明。如图 6 所示，原始训练数据是长度为 3 的序列”How can I”，其上下文具有正常的顺序依赖关系，因此注意力掩码为标准下三角矩阵。三个位置的输出分别为”are”、”we”、”do”，它们与”how”、”can”、”I”之间具有树形上下文关系。因此，当输入”are”、”we”、”do”进入第二步时，注意力掩码需要相应调整，如图 6 右上角所示。所有注意力掩码均为对角矩阵，但以原始训练数据为 key 时例外。此时使用矩阵乘法会造成大量计算浪费，因此我们可以通过向量点积仅计算对应位置的注意力分数。

HASS (Zhang et al., 2024) 和 EAGLE-3 均对注意力机制进行了类似的修改，以在训练中模拟测试过程，但这并非 EAGLE-3 的核心关注点。两者的动机、方法和结果截然不同。HASS 的动机是缓解 EAGLE 中特征预测不准确导致的误差积累。HASS 仍执行特征预测，包含特征预测损失 l_{fea} ，且草稿模型的输入必须是顶层特征。相比之下，EAGLE-3 的动机是去除不必要的约束以增强模型的表达能力。EAGLE-3 不再要求草稿模型的输出拟合目标模型的顶层特征，从而避免了误差积累。去除特征预测损失后，我们还能够发现此前从未发现的推理加速扩展规律。图 2 同样展示了 EAGLE-3 和 HASS 的加速效果，EAGLE-3 表现出显著更优的性能。

4 实验

模型。我们在最先进的开源对话和推理模型上进行实验，包括 Vicuna 13B (Chiang et al., 2023)、LLaMA-Instruct 3.1 8B、LLaMA-Instruct 3.3 70B (Dubey et al., 2024) 和 DeepSeek-R1-Distill-LLaMA 8B (DeepSeek-AI et al., 2025)。受 GPU 资源限制，我们无法在 405B 和 671B 模型上测试 EAGLE-3。

任务。遵循 EAGLE (Li et al., 2024c) 和 Spec-Bench (Xia et al., 2024) 的设置，我们在五项常见任务上进行评测，对所有任务使用相同权重，不在各任务上微调。多轮对话、代码生成、数学推理、指令遵循和摘要生成分别使用 MT-bench (Zheng et al., 2023)、HumanEval (Chen et al., 2021)、GSM8K (Cobbe et al., 2021)、Alpaca (Taori et al., 2023) 和 CNN/Daily Mail (Nallapati et al., 2016) 数据集。

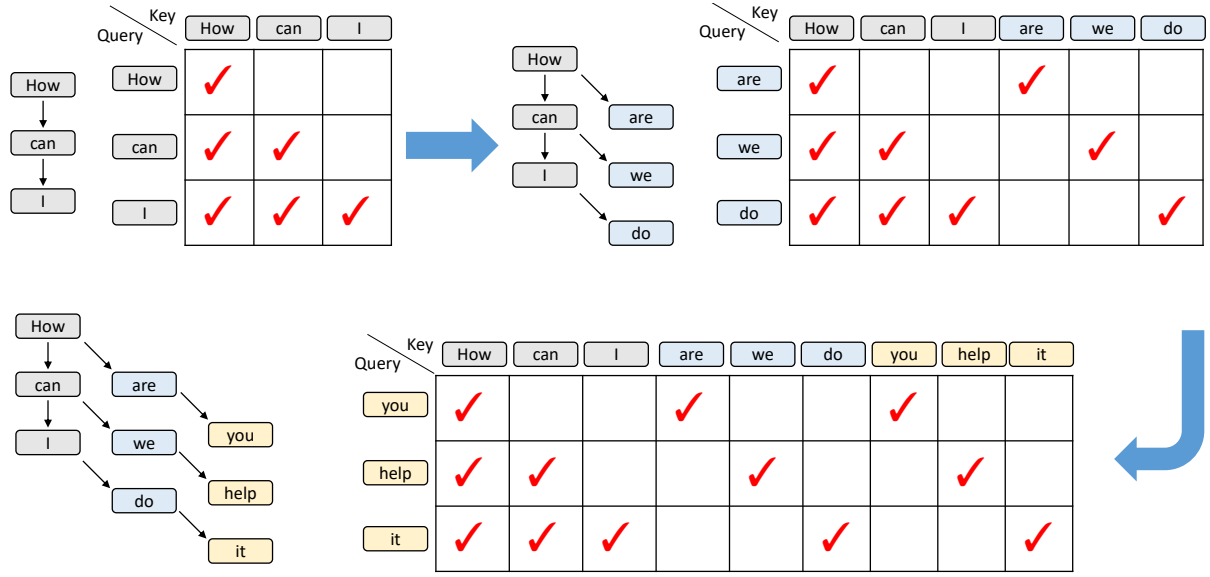


Figure 6: 训练时测试中注意力因果掩码的示意图。依次展示一个原生训练步骤（第一步）和两个模拟训练步骤（第二步和第三步）。词元之间的箭头表示上下文关系。灰色词元表示训练数据，蓝色和黄色词元分别表示草稿模型的第一轮和第二轮预测。

评测指标。 EAGLE-3 不修改目标模型的权重，使用严格的推测性采样接受条件，确保生成质量无损。因此，我们不评估生成质量，而是使用以下指标评估加速性能：

- **加速比：** 相对于原始自回归解码的实际测试加速比。
- **平均接受长度 τ ：** 每个草稿-验证循环平均生成的词元数，对应从草稿中被接受的词元数。
- **接受率 $n-\alpha$ ：** 草稿词元被接受的比例，直接反映草稿模型对目标模型的近似程度。遵循 EAGLE 的设置，在测试接受率时使用链式草稿而非树式草稿。EAGLE 存在误差积累问题，即草稿模型的输入可能是其自身的估计值而非目标模型的精确值。因此，EAGLE 用 $n-\alpha$ 表示输入包含 n 个估计特征时的接受率（条件为前面的估计词元均被目标模型接受）。换言之，即输入为 $f_1, f_2, \dots, f_i, \hat{f}_{i+1}, \dots, \hat{f}_{i+n}$ 时的接受率（ f 为精确值， $\hat{f}$ 为草稿模型的估计值）。类似地，我们用 $n-\alpha$ 表示 EAGLE-3 中输入包含 n 个自预测值 a 时的接受率，即输入为 $g_1, g_2, \dots, g_i, a_{i+1}, \dots, a_{i+n}$ 时的接受率（ g 为目标模型的融合特征）。

实现细节。 我们使用 AdamW 优化器， (β_1, β_2) 设置为 (0.9, 0.95)，并实施梯度裁剪（裁剪阈值为 0.5）。学习率设为 $5e-5$ 。训练数据使用 ShareGPT 和 UltraChat-200K (Ding et al., 2023)，分别包含约 68K 和 464K 条数

据。我们调用目标模型生成回复，而非使用固定数据集。对于推理模型 DeepSeek-R1-Distill-LLaMA 8B，我们还使用了 OpenThoughts-114k-math 数据集进行训练。

对比方法。 以原始自回归解码为基线 (1.00x)。我们将 EAGLE-3 与近期无损推测性采样方法进行对比，包括标准推测性采样 (Leviathan et al., 2023; Chen et al., 2023; Gante, 2023)、PLD (Saxena, 2023)、Medusa (Cai et al., 2024)、Lookahead (Fu et al., 2024)、Hydra (Ankner et al., 2024)、HASS (Zhang et al., 2024)、EAGLE (Li et al., 2024c) 和 EAGLE-2 (Li et al., 2024b)。

4.1 有效性分析

图 1 和表 1 展示了 EAGLE-3 的加速性能。在所有任务和目标模型上，EAGLE-3 均实现了最高的加速比和平均接受长度。EAGLE-3 相比原始自回归生成实现了约 3.0x–6.5x 的加速，比 EAGLE-2 提升了 20%–40%。不同任务对草稿模型接受率的影响不同，因此平均接受长度和加速比均与任务相关。由于代码生成任务中存在大量固定模板，草稿生成最为容易，因此 EAGLE-3 在 HumanEval 上表现最佳，加速比高达 6.5x，平均接受长度高达 7.5。DeepSeek-R1-Distill-LLaMA 8B 是个例外，其在数学推理数据集 GSM8K 上的加速比最高。这可能是因为使用了 OpenThoughts-114k-math 数据集训练 DeepSeek-R1-Distill-LLaMA 8B 的草稿模型。

图 7 展示了以 LLaMA-Instruct 3.1 8B 为

Table 1: 不同方法的加速比与平均接受长度 τ 。V 表示 Vicuna, L31 表示 LLaMA-Instruct 3.1, L33 表示 LLaMA-Instruct 3.3, DSL 表示 DeepSeek-R1-Distill-LLaMA。SpS 表示标准推测性采样, 其草稿模型为 Vicuna-68M。Medusa 等方法在非贪心设置下放宽了接受条件, 不保证无损加速, 因此在温度 =1 时不与 EAGLE-3 对比。

		MT-bench		HumanEval		GSM8K		Alpaca		CNN/DM		均值	
模型	方法	加速比	τ	加速比	τ	加速比	τ	加速比	τ	加速比	τ	加速比	τ
温度 =0													
V 13B	SpS	1.93x	2.27	2.23x	2.57	1.77x	2.01	1.76x	2.03	1.93x	2.33	1.92x	2.24
	PLD	1.58x	1.63	1.85x	1.93	1.68x	1.73	1.16x	1.19	2.42x	2.50	1.74x	1.80
	Medusa	2.07x	2.59	2.50x	2.78	2.23x	2.64	2.08x	2.45	1.71x	2.09	2.12x	2.51
	Lookahead	1.65x	1.69	1.71x	1.75	1.81x	1.90	1.46x	1.51	1.46x	1.50	1.62x	1.67
	Hydra	2.88x	3.65	3.28x	3.87	2.93x	3.66	2.86x	3.53	2.05x	2.81	2.80x	3.50
	EAGLE	3.07x	3.98	3.58x	4.39	3.08x	3.97	3.03x	3.95	2.49x	3.52	3.05x	3.96
	EAGLE-2	4.26x	4.83	4.96x	5.41	4.22x	4.79	4.25x	4.89	3.40x	4.21	4.22x	4.83
	EAGLE-3	5.58x	6.65	6.47x	7.54	5.32x	6.29	5.16x	6.17	5.01x	6.47	5.51x	6.62
L31 8B	EAGLE-2	3.16x	4.05	3.66x	4.71	3.39x	4.24	3.28x	4.12	2.65x	3.45	3.23x	4.11
	EAGLE-3	4.40x	6.13	4.85x	6.74	4.48x	6.23	4.82x	6.70	3.65x	5.34	4.44x	6.23
L33 70B	EAGLE-2	2.83x	3.67	3.12x	4.09	2.83x	3.69	3.03x	3.92	2.44x	3.55	2.85x	3.78
	EAGLE-3	4.11x	5.63	4.79x	6.52	4.34x	6.15	4.30x	6.09	3.27x	5.02	4.12x	5.88
DSL 8B	EAGLE-2	2.92x	3.80	3.42x	4.29	3.40x	4.40	3.01x	3.80	3.53x	3.33	3.26x	3.92
	EAGLE-3	4.05x	5.58	4.59x	6.38	5.01x	6.93	3.65x	5.37	3.52x	4.92	4.16x	5.84
温度 =1													
V 13B	SpS	1.62x	1.84	1.72x	1.97	1.46x	1.73	1.52x	1.78	1.66x	1.89	1.60x	1.84
	EAGLE	2.32x	3.20	2.65x	3.63	2.57x	3.60	2.45x	3.57	2.23x	3.26	2.44x	3.45
	EAGLE-2	3.80x	4.40	4.22x	4.89	3.77x	4.41	3.78x	4.37	3.25x	3.97	3.76x	4.41
	EAGLE-3	4.57x	5.42	5.15x	6.22	4.71x	5.58	4.49x	5.39	4.33x	5.72	4.65x	5.67
L31 8B	EAGLE-2	2.44x	3.16	3.39x	4.39	2.86x	3.74	2.83x	3.65	2.44x	3.14	2.80x	3.62
	EAGLE-3	3.07x	4.24	4.13x	5.82	3.32x	4.59	3.90x	5.56	2.99x	4.39	3.45x	4.92
L33 70B	EAGLE-2	2.73x	3.51	2.89x	3.81	2.52x	3.36	2.77x	3.73	2.32x	3.27	2.65x	3.54
	EAGLE-3	3.96x	5.45	4.36x	6.16	4.17x	5.95	4.14x	5.87	3.11x	4.88	3.95x	5.66
DSL 8B	EAGLE-2	2.69x	3.41	3.01x	3.82	3.16x	4.05	2.64x	3.29	2.35x	3.13	2.77x	3.54
	EAGLE-3	3.20x	4.49	3.77x	5.28	4.38x	6.10	3.16x	4.30	3.08x	4.27	3.52x	4.89

目标模型时, EAGLE 和 EAGLE-3 在 MT-bench 上的接受率。EAGLE-3 的接受率显著高于 EAGLE。随着草稿模型自身输入增多, EAGLE 的接受率急剧下降, 而 EAGLE-3 的接受率几乎保持不变, 验证了训练时测试技术的有效性。

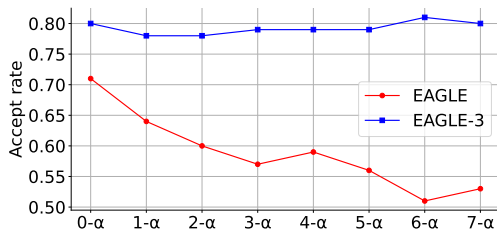


Figure 7: 以 LLaMA-Instruct 3.1 8B 为目标模型时, EAGLE 和 EAGLE-3 在 MT-bench 上的接受率。 $n-\alpha$ 表示在前面的估计词元均被目标模型接受的条件下, 输入包含 n 个估计特征时的接受率。

4.2 消融实验

EAGLE-3 的改进主要来自两个方面: 第一, 去除特征回归约束; 第二, 将仅复用顶层特征改为复用低层、中层和高层特征的融合。我们以 LLaMA-Instruct 3.1 8B 为目标模型在 MT-bench 上进行了消融实验。结果如表 2 所示, EAGLE-3 的两项改进均显著提升了接受长度和加速比, 验证了 EAGLE-3 设计的合理性。

Table 2: 以 LLaMA-Instruct 3.1 8B 为目标模型的消融实验结果。”去除特征约束”对应 EAGLE-3 的第一项改进, 即去除特征预测约束; ”融合特征”对应第二项改进, 即以低层、中层和高层特征融合取代顶层特征。

方法	MT-bench		GSM8K	
	加速比	τ	加速比	τ
EAGLE-2	3.16x	4.05	3.39x	4.24
+ 去除特征约束	3.82x	5.37	3.77x	5.22
+ 融合特征 (本方法)	4.40x	6.13	4.48x	6.23

4.3 EAGLE-3 在 SGLang 中的表现

EAGLE-3 等推测性采样算法通过利用冗余计算能力减少内存访问，从而降低内存受限解码阶段的延迟。随着批大小增加，这种冗余减少，推测性采样的效果也随之降低。在高度优化的生产级框架中提升效率更具挑战性。

SGLang 团队在单张 H100 GPU、LLaMA-Instruct 3.1 8B 以及 SGLang v0.4.4 环境 (Zheng et al., 2024) 下评测了 EAGLE-3 在大批大小时的性能，结果如表 3 所示。该实验未使用树形结构，链式长度设为 3，测试数据集为 MT-Bench。EAGLE 在批大小为 24 时吞吐量下降，而 EAGLE-3 在批大小为 64 时仍能实现 38% 的吞吐量提升。

Table 3: 在 H100 和 LLaMA-Instruct 3.1 8B 上 MT-Bench 数据集不同批大小下的吞吐量提升，以不使用推测性采样的 SGLang 为基线 (1.00x)。实验由 SGLang 团队进行。

批大小	2	4	8	16	24	32	48	56	64
EAGLE	1.40x	1.38x	1.23x	1.02x	0.93x	0.94x	0.88x	0.99x	0.99x
EAGLE-3	1.81x	1.82x	1.62x	1.48x	1.39x	1.32x	1.38x	1.34x	1.38x

SGLang 团队还测试了在 H100、目标模型为 LLaMA-Instruct 3.1 8B、测试数据集为 MT-bench 时，批大小 =1 下 EAGLE-3 的吞吐量，结果如表 4 所示。

Table 4: 在 H100、目标模型为 LLaMA-Instruct 3.1 8B、测试数据集为 MT-bench 时批大小 =1 下的吞吐量。实验由 SGLang 团队进行。

方法	吞吐量 (批大小 =1)
SGLang (无推测性采样, 1x H100)	158.34 词元/秒
SGLang + EAGLE-2 (1x H100)	244.10 词元/秒
SGLang + EAGLE-3 (1x H100)	373.25 词元/秒

4.4 EAGLE-3 在 vLLM 中的表现

我们还基于广泛使用的生产级框架 vLLM (Kwon et al., 2023)，研究了 EAGLE-3 对大批大小下吞吐量的影响，结果如表 5 所示 (RTX3090, LLaMA-Instruct 3.1 8B)。EAGLE 在批大小为 24 时吞吐量提升最大，而 EAGLE-3 在批大小为 56 时达到最大提升。该实验未使用树形结构，最大链式长度设为 2，测试数据集为 MT-Bench。

5 相关工作

已有许多方法被用于加速 LLM 推理，例如量化 (Hubara et al., 2018; Shen et al., 2020; Kim et al., 2021; Zadeh et al., 2020; Zafrir et al., 2019) 和知识蒸馏 (Hinton et al., 2015)。这些

Table 5: 在 A100 和 LLaMA-Instruct 3.1 8B 上 MT-Bench 数据集不同批大小下的吞吐量提升，以不使用推测性采样的 vLLM 为基线 (1.00x)。

批大小	2	4	8	16	24	32	48	56
EAGLE	1.30x	1.25x	1.21x	1.10x	1.03x	0.93x	0.82x	0.71x
EAGLE-3	1.75x	1.68x	1.58x	1.49x	1.42x	1.36x	1.21x	1.01x

方法通常存在权衡取舍，需要在模型性能与加速收益之间寻求平衡。

推测性采样使用目标模型进行验证，确保无损加速。早期推测性解码方法 (Stern et al., 2018; Sun et al., 2021) 在贪心设置下加速生成，而 Leviathan et al. (2023); Chen et al. (2023) 引入了推测性采样，将草稿-验证框架扩展至非贪心生成。此后涌现出大量改进工作。EAGLE (Li et al., 2024c)、EAGLE-2 (Li et al., 2024b)、Medusa (Cai et al., 2024) 和 Hydra (Ankner et al., 2024) 复用了目标模型的特征。HASS (Zhang et al., 2024) 在训练过程中模拟多步草稿过程，以缓解 EAGLE 中训练-推理不一致和误差积累问题。GLIDE 和 CAPE (Du et al., 2024) 复用目标模型的 KV 缓存，而 Draft & Verify (Zhang et al., 2023) 等方法 (Hooper et al., 2023; Yang et al., 2023; Monea et al., 2023; Li et al., 2024a; Yi et al., 2024; Liu et al., 2024b; Sun et al., 2024a; Elhoushi et al., 2024; Svirschevski et al., 2024) 利用层跳过或早退机制复用部分目标模型参数。

6 结论

本文提出了 EAGLE-3。在 EAGLE 的基础上，EAGLE-3 包含两项关键改进：第一，去除特征预测约束，通过训练时测试直接预测草稿词元；第二，以目标模型低层、中层和高层特征的融合取代仅使用顶层特征，从而获取更丰富的信息。在这些改进下，EAGLE-3 持续受益于训练数据的增加，实现了高达 6.5 倍的最大加速比。

致谢

感谢 James Liu、Ke Bao、Yineng Zhang、Lianmin Zheng、Ying Sheng 以及 SGLang 团队的许多其他成员，感谢他们将 EAGLE-3 合并至 SGLang 环境并进行评测。

References

Zachary Ankner, Rishab Parthasarathy, Anirudha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. Hydra: Sequentially-dependent draft heads

- for Medusa decoding. *arXiv preprint arXiv:2402.05109*.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv: 2401.10774*.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. [Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality](#).
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Daya Guo DeepSeek-AI, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*.
- Cunxiao Du, Jing Jiang, Xu Yuanchen, Jiawei Wu, Sicheng Yu, Yongqi Li, Shenggui Li, Kai Xu, Liqiang Nie, Zhaopeng Tu, et al. 2024. Glide with a cape: A low-hassle method to accelerate speculative decoding. *arXiv preprint arXiv:2402.02082*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. 2024. Layer skip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*.
- Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of LLM inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*.
- Joao Gante. 2023. Assisted generation: a new direction toward low-latency text generation. <https://huggingface.co/blog/assisted-generation>.
- Xiangxiang Gao, Weisheng Xie, Yiwei Xiang, and Feng Ji. 2024. Falcon: Faster and parallel inference of large language models through enhanced semi-autoregressive drafting and custom-designed decoding tree. *arXiv preprint arXiv:2412.12639*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Sophia Shao. 2023. Speed: Speculative pipelined execution for efficient decoding. *arXiv preprint arXiv:2310.12072*.
- Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2018. Quantized neural networks: Training neural networks with low precision weights and activations. *journal of machine learning research*, 18(187):1–30.
- Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2021. I-bert: Integer-only bert quantization. In *International conference on machine learning*, pages 5506–5518. PMLR.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Symposium on Operating Systems Principles*, pages 611–626.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

- Minghan Li, Xilun Chen, Ari Holtzman, Beidi Chen, Jimmy Lin, Wen-tau Yih, and Xi Victoria Lin. 2024a. Nearest neighbor speculative decoding for llm generation and attribution. *arXiv preprint arXiv:2405.19325*.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024b. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Conference on Empirical Methods in Natural Language Processing*.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024c. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024a. DeepSeek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Kai Han, and Yunhe Wang. 2024b. Kangaroo: Lossless self-speculative decoding via double early exiting. *arXiv preprint arXiv:2404.18911*.
- Giovanni Monea, Armand Joulin, and Edouard Grave. 2023. Pass: Parallel speculative sampling. *arXiv preprint arXiv:2311.13581*.
- Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. 2016. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023*.
- Apoorv Saxena. 2023. [Prompt lookup decoding](#).
- Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. 2020. Q-bert: Hessian based ultra low precision quantization of bert. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8815–8821.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31.
- Hanshi Sun, Zhuoming Chen, Xinyu Yang, Yuan-dong Tian, and Beidi Chen. 2024a. Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. *arXiv preprint arXiv:2404.11912*.
- Xin Sun, Tao Ge, Furu Wei, and Houfeng Wang. 2021. Instantaneous grammatical error correction with shallow aggressive decoding. *arXiv preprint arXiv:2106.04970*.
- Ziteng Sun, Jae Hun Ro, Ahmad Beirami, and Ananda Theertha Suresh. 2024b. Optimal block-level draft verification for accelerating speculative decoding. *arXiv preprint arXiv:2403.10444*.
- Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. 2024c. SpecTr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems*, 36.
- Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. 2024. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices. *arXiv preprint arXiv:2406.02532*.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>, 3(6):7.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutu Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*.
- Seongjun Yang, Gibbeum Lee, Jaewoong Cho, Dimitris Papailiopoulos, and Kangwook Lee. 2023. Predictive pipelined decoding: A compute-latency trade-off for exact llm decoding. *arXiv preprint arXiv:2307.05908*.
- Hanling Yi, Feng Lin, Hongbin Li, Peiyang Ning, Xiaotian Yu, and Rong Xiao. 2024. Generation meets verification: Accelerating large language model inference with smart parallel auto-correct decoding. *arXiv preprint arXiv:2402.11809*.
- Ali Hadi Zadeh, Isak Edo, Omar Mohamed Awad, and Andreas Moshovos. 2020. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 811–824. IEEE.

- Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pages 36–39. IEEE.
- Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2023. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*.
- Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. 2024. Learning harmonized representations for speculative sampling. *arXiv preprint arXiv:2408.15766*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot Arena. *arXiv preprint arXiv:2306.05685*.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37:62557–62583.

A 实现细节

原始方法： 我们使用 Huggingface Transformers 库中的模型，采用 PyTorch 后端并预分配 KV 缓存。其他方法同样以这些模型为基础。

标准推测性采样： 我们使用 HuggingFace Transformers 库的辅助生成功能。

PLD、Lookahead、Medusa 和 Hydra： 我们使用默认设置和官方发布的权重。

EAGLE： Vicuna 和 LLaMA2-Chat 草稿模型使用官方发布权重，而 LLaMA3-Instruct 使用 ShareGPT 数据集训练（与 Medusa 和 Hydra 保持一致）。

EAGLE-2： 对于 7B (8B)、13B 和 70B 规模的原始 LLM，我们分别将草稿词元总数设为 60、50 和 48，草稿树深度为 6，扩展阶段选择 10 个节点。

EAGLE-3： EAGLE-3 的草稿模型具有显著更高的接受率，使我们能够在保持节点数量与 EAGLE-2 相同的情况下，将草稿树深度从 6 增加到 8。